```
s = "".html_safe
s.html_safe? # => true
```

It is important to understand that `html_safe` performs no escaping whatsoever, it is just an assertion:

```
s = "<script>...</script>".html_safe
s.html_safe? # => true
s            # => "<script>...</script>"
```

It is your responsibility to ensure calling `html_safe` on a particular string is fine.

If you append onto a safe string, either in-place with `concat`/`<<`, or with `+`, the result is a safe string.

Unsafe arguments are escaped:

```
"".html_safe + "<" # => "&lt;"
```

Safe arguments are directly appended:

```
"".html_safe + "<".html_safe # => "<"
```

These methods should not be used in ordinary views. Unsafe values are automatically escaped:

```
<%= @review.title %> <%# fine, escaped if needed %>
```

To insert something verbatim use the `raw` helper rather than calling `html_safe`:

```
<%= raw @cms.current_template %> <%# inserts @cms.current_template as
is %>
```

or, equivalently, use `<%==`:

```
<%== @cms.current_template %> <%# inserts @cms.current_template as is
%>
```

The `raw` helper calls `html_safe` for you:

```
def raw(stringish)
  stringish.to_s.html_safe
end
```

Defined in `active_support/core_ext/string/output_safety.rb`.

5.1.3 Transformation

As a rule of thumb, except perhaps for concatenation as explained above, any method that may change a string gives you an unsafe string. These are `downcase`, `gsub`, `strip`, `chomp`, `underscore`, etc.

In the case of in-place transformations like `gsub!` the receiver itself becomes unsafe.

The safety bit is lost always, no matter whether the transformation actually changed something.

5.1.4 Conversion and Coercion

Calling `to_s` on a safe string returns a safe string, but coercion with `to_str` returns an unsafe string.

5.1.5 Copying

Calling `dup` or `clone` on safe strings yields safe strings.

5.2 `remove`

The method `remove` will remove all occurrences of the pattern:

```
"Hello World".remove(/Hello /) # => "World"
```

There's also the destructive version `String#remove!`.

Defined in `active_support/core_ext/string/filters.rb`.

5.3 `squish`

The method `squish` strips leading and trailing whitespace, and substitutes runs of whitespace with a single space each:

```
" \n  foo\n\r \t bar \n".squish # => "foo bar"
```

There's also the destructive version `String#squish!`.

Note that it handles both ASCII and Unicode whitespace.